

PYTHON FOR EVERYBODY

A Comprehensive Learning Companion

Course 1

Getting Started with Python

Variables · Expressions · Conditionals · Functions · Loops

Intermediate Level · Python 3 · Comprehensive Theory & Hands-on Coding

CHAPTER 1

Getting Started with Python

Foundations of the Language, Syntax, and Core Programming Constructs

COURSE OVERVIEW

This chapter provides a rigorous, structured treatment of Python's foundational constructs as taught in

Course 1 of Python for Everybody. The content is organized into five major topic areas, each mirroring

a chapter from the original course: Introduction to Python, Expressions & Variables, Conditional Statements, Functions, and Loops & Iteration.

Section 1 (Theory) presents every concept with precise definitions, contextual explanations, and comparative tables. Section 2 (Practical & Coding) presents complete, well-commented IPython notebook

cells with comprehensive line-by-line explanations for every statement.

Prerequisites: Basic comfort with a text editor or Jupyter Notebook. No prior programming experience required.

SECTION 1 — THEORETICAL FOUNDATIONS

1.1 Introduction to Python Programming

Python is a high-level, general-purpose, interpreted programming language first created by Guido van Rossum and officially released in 1991. Its name, contrary to popular assumption, is not derived from the snake — it was named after the British comedy troupe Monty Python's Flying Circus, reflecting the language's emphasis on readability and the enjoyment of programming.

Python occupies a distinctive position in the ecosystem of programming languages. It is simultaneously lightweight — in the sense that it imposes minimal syntactic overhead on the programmer — and versatile, being applicable to domains ranging from web development and data science to artificial intelligence, scientific computing, and system automation.

1.1.1 Conceptualizing a Programming Language

To understand Python as a language, it is instructive to draw an analogy to natural human language. Just as English or Hindi possesses vocabulary, grammatical rules, and narrative structure, so too does Python operate on three analogous levels:

- **Vocabulary (Words):** In Python, the vocabulary consists of variables — programmer-defined named containers for data — and reserved words, which are terms that carry fixed, predefined meaning within the language and cannot be repurposed by the programmer.
- **Sentence Structure (Syntax):** Valid syntax patterns govern how words may be combined to form meaningful instructions. A Python statement such as `x = x + 2` is syntactically valid; `x + = 2 x` is not. The interpreter enforces these rules strictly.
- **Story Structure (Program):** A complete Python program is an organized collection of syntactically valid statements arranged to accomplish a specific, meaningful goal. The structure of the program — how statements are sequenced, grouped, and conditionally executed — determines its behaviour.

Why Python?

Python's design philosophy, articulated in 'The Zen of Python' (PEP 20), prioritizes readability and simplicity.

A Python program expressing a given idea typically requires significantly fewer lines of code than an equivalent

program written in Java or C++. This productivity advantage, combined with Python's rich ecosystem of libraries,

makes it the dominant language for data science, machine learning, and rapid application development.

1.1.2 Reserved Words (Keywords)

Reserved words — also called keywords — are identifiers that the Python interpreter has assigned specific syntactic meaning. They form the grammatical backbone of the language. Because the interpreter expects these words to appear in specific syntactic positions and to trigger specific behaviours, they cannot be used as variable names or any other programmer-defined identifier. Attempting to do so will result in a `SyntaxError`.

Python 3 defines 35 reserved words in total:

<code>False</code>	<code>None</code>	<code>True</code>	<code>and</code>	<code>as</code>
<code>assert</code>	<code>async</code>	<code>await</code>	<code>break</code>	<code>class</code>
<code>continue</code>	<code>def</code>	<code>del</code>	<code>elif</code>	<code>else</code>

<code>except</code>	<code>finally</code>	<code>for</code>	<code>from</code>	<code>global</code>
<code>if</code>	<code>import</code>	<code>in</code>	<code>is</code>	<code>lambda</code>
<code>nonlocal</code>	<code>not</code>	<code>or</code>	<code>pass</code>	<code>raise</code>
<code>return</code>	<code>try</code>	<code>while</code>	<code>with</code>	<code>yield</code>

⚠ Important Rule

You cannot use any of the 35 reserved words above as variable names, function names, or any other programmer-defined identifier. For example: `'for = 10'` will immediately raise a `SyntaxError`. These words are exclusively owned by the Python interpreter for its own grammatical purposes.

1.1.3 Sentences and Lines: The Building Blocks of a Python Program

In Python, the fundamental unit of executable instruction is the statement — analogous to a sentence in natural language. Each statement occupies one or more lines and instructs the interpreter to perform a specific action. There are four primary syntactic elements from which statements are constructed:

Element	Definition & Example
Variable	A named storage location in memory. The variable <code>x</code> in <code>x = 2</code> holds the value 2.
Operator	A symbol that performs an operation on one or more operands. Examples: <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>**</code> .
Constant	A fixed, literal value that does not change — e.g., the integer 2 or the string <code>'hello'</code> .
Function	A named, reusable block of code invoked by its name followed by parentheses — e.g., <code>print(x)</code> .

The three most common types of statements encountered in introductory Python are:

- **Assignment Statement:** Associates a variable name with a computed value. Example: `x = 2`. After execution, the name `x` refers to the integer object 2 in memory.
- **Assignment with Expression:** Evaluates an arithmetic or logical expression on the right-hand side and assigns the result to the variable on the left. Example: `x = x + 2`. The right-hand side is evaluated first, then the result overwrites the existing value of `x`.
- **Function Call Statement:** Invokes a named function, optionally passing arguments. Example: `print(x)`. The interpreter transfers control to the function's code block, which executes and returns.

1.1.4 Interactive vs. Script Mode

Python can be operated in two fundamentally different execution modes, each suited to different use cases:

- **Interactive Mode (REPL — Read-Eval-Print Loop):** The programmer types one statement at a time directly into the Python interpreter prompt (`>>>`). The interpreter reads the statement, evaluates it, prints the result if any, and awaits the next statement. This mode is ideal for exploratory computation, rapid prototyping, and learning — the feedback is immediate. The standard Python REPL and Jupyter Notebook cells operate in this mode.
- **Script Mode:** The programmer writes a complete sequence of statements into a plain-text file (conventionally with a `.py` extension) using a text editor or IDE. The interpreter is then invoked to execute all statements in the file sequentially. Script mode is appropriate for programs of non-trivial complexity, reproducible workflows, and any code intended for deployment or sharing.

1.2 Expressions and Variables

The manipulation of data through expressions and its storage in variables constitute the most elementary operations in any program. This section examines constants, variables, operators, type systems, and type conversion with the precision required for effective Python programming.

1.2.1 Constants

A constant is a literal data value whose value is fixed at the time it is written and does not change during program execution. Constants are not associated with a named memory location — they are the raw values themselves.

Type of Constant	Description & Examples
Integer Constant	Whole numbers, positive or negative, without a decimal point. Examples: 0, 42, -7, 1000000
Float Constant	Numbers with a fractional component, represented in decimal notation. Examples: 3.14, -0.001, 2.0, 1.5e10
String Constant	A sequence of characters enclosed in single quotes or double quotes. Examples: 'hello', "Python", 'Data Science 2024'
Boolean Constant	One of exactly two truth values: True or False (note the capitalised first letter — they are reserved words).
None Constant	A special singleton constant representing the intentional absence of a value. Used to initialise variables whose value is not yet known.

1.2.2 Variables

A variable is a named reference to a location in the computer's memory where a data value is stored. The name serves as a human-readable handle through which the stored value can be retrieved and, if necessary, replaced. In Python, variables are dynamically typed: the same variable name can reference objects of different types at different points during program execution.

The naming rules for Python variables are as follows:

1. A variable name must begin with either a letter (a–z, A–Z) or an underscore (_). It cannot begin with a digit.
2. After the first character, a variable name may consist of any combination of letters, digits (0–9), and underscores.
3. Variable names are case-sensitive: `score`, `Score`, and `SCORE` are three distinct identifiers.
4. A variable name must not coincide with any of Python's 35 reserved words.

Category	Examples
Valid names	<code>spam</code> <code>eggs</code> <code>spam23</code> <code>_speed</code> <code>total_count</code> <code>user_age_2</code>
Invalid names	<code>23spam</code> (starts with digit) <code>#sign</code> (invalid character) <code>var.12</code> (dot not allowed)
Case distinction	<code>spam</code> <code>Spam</code> <code>SPAM</code> — these are three completely different variables

A widely adopted best practice in Python is the use of mnemonic variable names — names whose meaning is self-evident from context. For example, `hours_worked`, `hourly_rate`, and `gross_pay` are far more informative than `h`, `r`, and `g` respectively. Mnemonic naming reduces cognitive overhead when reading code and is considered a hallmark of professional-quality Python.

1.2.3 Assignment Statements

The assignment statement is the primary mechanism for binding a value to a variable name. It is written using the equals sign (`=`), which in Python functions as the assignment operator — not a mathematical equality. The semantics are directional: the expression on the right-hand side is fully evaluated first, and the resulting value is then bound to the variable name on the left-hand side.

Critical Distinction

In mathematics, `'='` denotes equality between two expressions (a symmetric relationship).

In Python, `'='` denotes assignment (a directional operation).

The statement `x = x + 1` is mathematically nonsensical but programmatically valid:

it reads 'evaluate `(x + 1)`, then store the result in `x`, overwriting whatever was there before.'

1.2.4 Numeric Operators and Precedence

Python provides six arithmetic operators for numeric computation:

Operator	Operation & Notes
+	Addition — adds two numeric values. Also concatenates strings.
-	Subtraction — subtracts the right operand from the left.
*	Multiplication — multiplies two numeric values. Also repeats strings.
/	Division — always returns a floating-point result in Python 3 (e.g., $7 / 2 \rightarrow 3.5$).
**	Exponentiation (Power) — raises the left operand to the power of the right (e.g., $2 ** 8 \rightarrow 256$).
%	Remainder (Modulo) — returns the remainder after integer division (e.g., $17 \% 5 \rightarrow 2$).
//	Floor Division — returns the integer quotient, discarding any fractional part (e.g., $7 // 2 \rightarrow 3$).

When an expression contains multiple operators, Python resolves ambiguity by applying operator precedence rules, from highest to lowest priority:

5. Parentheses — expressions inside parentheses are always evaluated first, regardless of the operators they contain.
6. Exponentiation (**) — evaluated right-to-left when chained ($2 ** 3 ** 2 = 2 ** 9 = 512$).
7. Multiplication, Division, Floor Division, and Remainder (*, /, //, %) — evaluated left-to-right when multiple operators of this tier appear.
8. Addition and Subtraction (+, -) — evaluated left-to-right.
9. Left-to-right evaluation resolves ties among operators at the same precedence level.

Memory Aid (PEMDAS / BODMAS)

Parentheses → Exponents (Power) → Multiplication/Division → Addition/Subtraction

When in doubt, use parentheses to make your intended evaluation order explicit.

Example: $2 + 3 * 4$ evaluates to 14 (not 20), because * has higher precedence than +.

$(2 + 3) * 4$ evaluates to 20, because the parentheses force addition first.

1.2.5 Data Types and Type Conversion

Every value in Python belongs to a specific data type, which determines what operations can be performed on it and how it is stored in memory. Understanding data types is essential because Python

enforces strict rules about type compatibility — for example, you cannot add an integer to a string without first converting one type to the other.

The `type()` built-in function returns the data type of any value or variable: `type(42) → <class 'int'>`
`type(3.14) → <class 'float'>` `type('hello') → <class 'str'>`

Type	Description & Characteristics
<code>int</code>	Integer type. Whole numbers of arbitrary magnitude. Examples: 0, -42, 1_000_000. No decimal point.
<code>float</code>	Floating-point type. Numbers with a fractional component, stored in IEEE 754 double-precision format. Example: 3.14, -0.001, 2.0.
<code>str</code>	String type. An immutable sequence of Unicode characters. Delimited by single (') or double (") quotes. Example: 'Python', "hello world".
<code>bool</code>	Boolean type. Exactly two instances: True and False (capital first letter). A subclass of int in Python (True == 1, False == 0).
<code>NoneType</code>	The type of the singleton constant None, which represents the absence of a value.

Type Conversion Functions

Python provides built-in functions for explicit type conversion (also called type casting):

- `int(x)`: Converts x to an integer. If x is a float, it truncates (not rounds) the fractional part. If x is a string containing a valid integer representation, it converts it. Raises `ValueError` if the string contains non-numeric characters.
- `float(x)`: Converts x to a floating-point number. Accepts integers and strings representing valid numbers.
- `str(x)`: Converts x to its string representation. Works on virtually any Python object.

⚠ Integer Division: Python 2 vs Python 3

This is a historically significant difference between the two major Python versions:

Python 2: `7 / 2 → 3` (floor division — the fractional part is silently discarded)

Python 3: `7 / 2 → 3.5` (true division — a float result is always returned)

In Python 3, use `//` for floor division: `7 // 2 → 3`

All modern code should be written in Python 3, which is the current and maintained version.

1.2.6 The `input()` Function

The `input()` function is Python's primary mechanism for accepting data from the user at runtime. When executed, it pauses the program, displays an optional prompt string to the user, waits for the user to type a value and press Enter, and then returns the typed value as a string object.

This last point is of critical importance: `input()` always returns a string, regardless of what the user types. If the user types the number 42, `input()` returns the string `'42'`, not the integer 42. If the program intends to use the input value for arithmetic, it must explicitly convert it using `int()` or `float()` before performing any numeric operations.

1.2.7 Comments

A comment is an annotation within source code that is entirely ignored by the Python interpreter during execution. Its sole purpose is to communicate information to human readers — explaining the intent behind a section of code, documenting assumptions, or temporarily disabling a statement during debugging.

In Python, a comment is initiated by the hash symbol (`#`). Everything from the `#` character to the end of the line is treated as a comment. Comments may appear on their own line (inline with the code indentation level) or at the end of a statement line. There is no native multi-line comment syntax in Python; multi-line comments are conventionally written as consecutive single-line comments.

Best Practice: Self-Documenting Code

Effective comments explain WHY, not WHAT. The code itself should be clear enough to show what it does;

the comment should explain the reasoning, context, or non-obvious implication.

Poor comment: `x = x + 1 # add 1 to x`

Good comment: `x = x + 1 # increment retry counter before re-attempting the API call`

1.3 Conditional Statements

Conditional statements are the mechanism through which a program's execution path is determined at runtime based on the truth value of logical conditions. They represent the first form of non-sequential control flow — the ability for a program to make decisions.

1.3.1 Indentation: Python's Scope Delimiter

Before examining conditional statements, it is essential to understand Python's indentation convention, as it is syntactically mandatory — not merely stylistic. In most programming languages

(Java, C, C++, JavaScript), the scope of a code block is delimited by curly braces {}. Python replaces this convention entirely with consistent indentation.

In Python, a block of code that belongs to (is 'owned by') a conditional statement, loop, or function is written with a consistent indentation of four spaces (the PEP 8 standard) relative to the header line. When the indented block ends and the indentation returns to the previous level, Python recognizes this as the end of the block. Inconsistent indentation is a syntax error.

Indentation Is Not Optional

Python's use of indentation to define scope is one of its most distinctive syntactic features. Forgetting to indent after an 'if', 'for', 'while', or 'def' statement is an `IndentationError`. Mixing tabs and spaces within the same file will cause a `TabError`. Use 4 spaces per indentation level consistently throughout your code (PEP 8 standard).

1.3.2 The if Statement

The if statement evaluates a boolean expression (the condition). If the condition evaluates to `True`, the indented block of statements that follows is executed. If the condition evaluates to `False`, the indented block is skipped entirely, and execution resumes at the next de-indented statement.

The canonical structure of an if statement is:

```
if <boolean_expression>:
    <statement_to_execute_if_true>
    <another_statement_if_true>
<statement_always_executed>    # outside the if block
```

The colon (:) at the end of the if line is mandatory. It signals to the interpreter that an indented block follows.

1.3.3 Comparison Operators

Conditions in if statements are typically constructed using comparison operators, which evaluate to boolean values (`True` or `False`):

Operator	Meaning & Example
<	Less than. Returns <code>True</code> if left operand is numerically smaller than right. Example: <code>3 < 5 → True</code>
<=	Less than or equal to. Returns <code>True</code> if left is smaller than or equal to right. Example: <code>5 <= 5 → True</code>

Operator	Meaning & Example
<code>==</code>	Equal to. Checks value equality (not identity). Example: <code>5 == 5</code> → True. Note: <code>==</code> is comparison; <code>=</code> is assignment.
<code>>=</code>	Greater than or equal to. Example: <code>7 >= 10</code> → False
<code>></code>	Greater than. Example: <code>10 > 7</code> → True
<code>!=</code>	Not equal to. Returns True if the two operands have different values. Example: <code>3 != 5</code> → True

1.3.4 The if-else Statement (Two-Way Decision)

The if-else statement extends the simple if statement by providing an alternative code block that executes when the condition is False. This enables two-way branching: exactly one of the two blocks will always execute, guaranteeing that every execution path is handled.

```
if <condition>:
    <execute_when_condition_is_True>
else:
    <execute_when_condition_is_False>
<always_executes>
```

Note that the else clause has no condition of its own — it is simply the catch-all for when the if condition is not satisfied.

1.3.5 The elif Clause (Multi-Way Decision)

The elif (else if) clause enables multi-way branching: the evaluation of multiple conditions in sequence. Python evaluates each condition from top to bottom. The first condition that evaluates to True causes its associated block to execute; all subsequent elif and else clauses are then skipped. If no condition is True and an else clause is present, the else block executes.

```
if <condition_1>:
    <block_1>
elif <condition_2>:
    <block_2>
elif <condition_3>:
    <block_3>
else:
    <default_block>
<always_executes>
```

A crucial property of the if-elif-else chain is mutual exclusivity: because Python stops evaluating as soon as one True condition is found, only one block can ever execute per traversal through the chain, regardless of how many conditions might theoretically be True.

1.3.6 Nested Conditional Statements

A nested conditional is a conditional statement that appears within the indented block of another conditional statement. Nesting enables the expression of compound logical conditions where the relevance of a second condition depends entirely on the outcome of the first.

While nesting is occasionally the most natural way to express a logic structure, excessive nesting (more than two or three levels) rapidly degrades code readability. In practice, deeply nested conditionals are often refactored using elif chains, logical operators (and, or, not), or by extracting the inner logic into separate functions.

1.3.7 Exception Handling: try-except

The try-except construct is Python's primary mechanism for handling runtime errors (exceptions) gracefully. Without exception handling, an error during execution — such as attempting to convert the string 'hello' to an integer — produces a traceback and terminates the program immediately.

The try-except construct intercepts specified error conditions and directs program execution to a recovery block:

```
try:
    <statements_that_might_raise_an_exception>
except:
    <statements_to_execute_if_an_exception_occurred>
<continues_execution_regardless>
```

The semantics are as follows: Python attempts to execute the statements in the try block. If any statement raises an exception, execution of the try block is immediately halted and control is transferred to the except block. If no exception occurs, the except block is skipped entirely, and execution continues after the try-except structure.

1.4 Functions

A function is a named, self-contained block of code that encapsulates a specific, reusable computation. The ability to define and call functions is one of the most powerful mechanisms in any

programming language, as it enables the practitioner to decompose complex programs into manageable, independently testable components — each with a clearly defined responsibility.

1.4.1 The Principle of Abstraction through Functions

Functions embody the principle of abstraction: the hiding of implementation details behind a named interface. Once a function is defined, the caller need not understand how it accomplishes its task — only what inputs it requires and what output it produces. This separation of interface from implementation is foundational to software engineering at all levels of complexity.

Python provides a rich library of built-in functions that are available without any import statement: `print()`, `input()`, `type()`, `len()`, `max()`, `min()`, `int()`, `float()`, `str()`, `abs()`, `round()`, and many others. These functions represent prepackaged computations that would otherwise require non-trivial code to implement from scratch.

1.4.2 Defining a Function with `def`

Programmer-defined functions are created using the `def` keyword. The `def` statement does not execute the function's code — it defines it, registering the function under the specified name so that it can be invoked later. The function's code block, delimited by indentation, is stored but not executed at definition time.

```
def function_name(parameter_1, parameter_2, ...):  
    <statement_1>  
    <statement_2>  
    return <value>    # optional
```

The parentheses after the function name are mandatory, even if the function accepts no parameters. The colon terminates the header line, and all subsequent indented lines constitute the function body.

1.4.3 Parameters and Arguments

An important terminological distinction applies here:

- **Parameter:** A variable name declared in the function's `def` statement, within the parentheses. It acts as a local placeholder within the function body for the value that will be supplied at call time.
- **Argument:** The actual value supplied to the function when it is called. The argument is bound to the corresponding parameter for the duration of the function's execution.

Example: In `def greet(name):`, the word `name` is a parameter. In the call `greet('Alice')`, the string `'Alice'` is the argument. During the execution of `greet()`, the parameter `name` holds the value `'Alice'`.

1.4.4 The `return` Statement

The `return` statement serves two functions simultaneously:

10. It transmits a value back to the caller — the expression following `return` is evaluated, and the resulting value becomes the value of the function call expression in the caller's context.
11. It terminates the function's execution immediately. Any statements following the `return` statement within the function body are not executed.

A function that reaches its end without encountering a `return` statement, or that contains a bare `return` statement with no expression, implicitly returns `None`. This is a common source of subtle bugs: assigning the result of such a function to a variable and then attempting to use it in arithmetic will raise a `TypeError`.

1.5 Loops and Iteration

Loops are control flow structures that enable the repeated execution of a block of code — either for a known number of times or until a specified condition is no longer satisfied. Iteration is the general term for the process of stepping through a sequence of values or repeatedly executing a block of statements.

1.5.1 Indefinite Loops: `while`

The `while` loop is classified as an indefinite loop because the number of iterations is not determined in advance — it depends entirely on when the loop's condition transitions from `True` to `False` during execution.

```
while <condition>:
    <loop_body>      # executes repeatedly as long as condition is True
<post_loop_code>    # executes once, after condition becomes False
```

Execution semantics: Before each iteration, the condition is evaluated. If `True`, the body executes and the condition is re-evaluated. If `False`, the loop terminates and execution proceeds to the first statement after the loop. If the condition is `False` on the very first evaluation, the body never executes — this is the zero-trip loop.

Infinite Loops

If the condition of a `while` loop never becomes `False`, the loop runs forever (an infinite loop). This is almost always a programming error. It is caused by failing to update the variable(s)

that the condition depends on within the loop body.

Example of infinite loop: `n = 5`

```
while n > 0: # n never changes — loops forever
    print('hello')
```

Prevention: Always ensure the loop body contains a statement that will eventually make the condition False.

1.5.2 Loop Control: break and continue

Python provides two keywords for fine-grained control over loop iteration:

- **break:** Immediately terminates the innermost enclosing loop, regardless of the loop condition. Execution jumps to the first statement after the loop. `break` is commonly used to exit a `while True` loop when a termination condition is detected inside the loop body.
- **continue:** Immediately terminates the current iteration and jumps back to the loop header, where the condition is re-evaluated (for `while`) or the next item is fetched (for `for`). Statements in the loop body that follow the `continue` statement are not executed for that iteration.

Statement	Effect & Analogy
<code>break</code>	Exits the loop entirely. Like closing a book mid-chapter and not returning to it.
<code>continue</code>	Skips the rest of the current iteration and starts the next one. Like skipping a page but continuing to read the book.

1.5.3 Definite Loops: for

The `for` loop is a definite loop: it iterates over each element of an iterable object (such as a list, string, range, or tuple), executing the loop body once for each element. The total number of iterations is determined in advance by the length of the iterable.

```
for <iteration_variable> in <iterable>:
    <loop_body>
<post_loop_code>
```

At each iteration, the iteration variable is automatically assigned the next value from the iterable. The programmer does not need to manage an index or manually update the variable — Python handles this automatically. When all elements have been processed, the loop terminates normally.

The `range()` function is frequently used with for loops to generate a sequence of integers: `range(5)` produces 0, 1, 2, 3, 4. `range(1, 6)` produces 1, 2, 3, 4, 5. `range(0, 10, 2)` produces 0, 2, 4, 6, 8 (step of 2).

1.5.4 Common Loop Patterns

Several canonical computational patterns are routinely implemented using loops. Each pattern addresses a common class of problem:

Pattern	Description & Implementation Strategy
Counting	Counting the number of elements in a collection that satisfy a condition. Initialize a counter variable to 0. Increment by 1 inside the loop whenever the condition is met.
Summation	Accumulating a running total of all values in a collection. Initialize a sum variable to 0. Add each element to the sum inside the loop.
Finding the Maximum	Tracking the largest value encountered so far. Initialize to the smallest plausible value or -1. Inside the loop, update if the current element is larger.
Finding the Minimum	Tracking the smallest value seen so far. Initialize to None. Inside the loop, set to current value if None (first iteration) or if smaller than current minimum.
Boolean Search	Determining whether a target value exists in a collection. Initialize a boolean flag to False. Set to True inside the loop when the target is found.

SECTION 2 — PRACTICAL IMPLEMENTATION — JUPYTER NOTEBOOK CELLS

This section presents a complete, executable IPython Notebook covering all topics introduced in Section 1. Each cell is presented with the complete code followed by a comprehensive line-by-line explanation. The cells are designed to be executed in sequence in a Jupyter Notebook environment.

Setup

Environment: Python 3.10+ | Jupyter Notebook or JupyterLab

Installation: `pip install jupyter`

Launch: `jupyter notebook`

All cells below can be pasted sequentially into a Jupyter Notebook and executed in order.

Cell 1: Variables, Constants, and Assignment

This cell introduces the most fundamental operations in Python: declaring variables, assigning values of different types, and observing how Python tracks type information.

```
# — Cell 1: Variables, Constants, and Assignment —————

# --- 1.1: Integer constants and assignment ---
age = 25                                # Line (1)
year_of_birth = 2025 - age              # Line (2)
negative_number = -42                   # Line (3)

# --- 1.2: Float constants ---
pi = 3.14159265                         # Line (4)
temperature_celsius = -12.5             # Line (5)

# --- 1.3: String constants ---
first_name = 'Arjun'                    # Line (6)
last_name = "Sharma"                    # Line (7)
full_name = first_name + ' ' + last_name # Line (8)

# --- 1.4: Boolean constants ---
is_student = True                       # Line (9)
has_experience = False                   # Line (10)
```

```

# --- 1.5: None constant ---
salary = None                                # Line (11)

# --- 1.6: Checking types ---
print(type(age))                             # Line (12)
print(type(pi))                             # Line (13)
print(type(first_name))                     # Line (14)
print(type(is_student))                     # Line (15)
print(type(salary))                         # Line (16)

# --- 1.7: Displaying values ---
print('Full name:', full_name)              # Line (17)
print('Year of birth:', year_of_birth)      # Line (18)

# --- Expected Output ---
# <class 'int'>
# <class 'float'>
# <class 'str'>
# <class 'bool'>
# <class 'NoneType'>
# Full name: Arjun Sharma
# Year of birth: 2000

```

Line-by-Line Explanation — Cell 1

Code Line	Explanation
# Line (1) age = 25	Declares a variable named age and assigns it the integer constant 25. Python allocates memory, stores the integer object 25, and binds the name 'age' to that object. The integer type is inferred automatically.
# Line (2) year_of_birth = ...	Evaluates the arithmetic expression (2025 - 25 = 2000) on the right-hand side first, then assigns the resulting integer 2000 to the new variable year_of_birth. This demonstrates that expressions are fully evaluated before assignment occurs.
# Line (3) negative_number = -42	Assigns a negative integer constant. Python integers are signed and can represent any negative or positive whole number without bound (unlike fixed-width integers in C or Java).
# Line (4) pi = 3.14159265	Assigns a floating-point constant to the variable pi. The presence of the decimal point instructs Python to create a float object. Python uses 64-bit IEEE 754 double-precision floating-point representation internally.
# Line (5) temperature_celsius = -12.5	Assigns a negative float. Demonstrates that float constants may be negative and that the variable naming convention uses underscores to separate words (snake_case) — the Python standard.

Code Line	Explanation
# Line (6) first_name = 'Arjun'	Assigns a string constant enclosed in single quotes. Python creates an immutable str object containing the six Unicode characters A, r, j, u, n. Single and double quotes are interchangeable for simple string literals.
# Line (7) last_name = "Sharma"	Assigns a string constant enclosed in double quotes. Functionally identical to using single quotes. Double quotes are preferred when the string itself contains apostrophes (e.g., "it's fine").
# Line (8) full_name = first_name + ' ' + last_name	Demonstrates string concatenation using the + operator. Python evaluates left-to-right: ('Arjun') + (' ') → ('Arjun '), then ('Arjun ') + ('Sharma') → ('Arjun Sharma'). The resulting string is assigned to full_name. Note: the + operator on strings performs concatenation, not addition.
# Line (9) is_student = True	Assigns the boolean constant True to is_student. Note the capital T — this is mandatory. true (lowercase) is not a reserved word in Python; it would be interpreted as an undefined variable name.
# Line (10) has_experience = False	Assigns the boolean constant False. Boolean variables are conventionally named with verb phrases (is_, has_, can_, should_) to make their True/False semantics self-evident.
# Line (11) salary = None	Assigns the special singleton constant None to salary. This communicates that salary is a variable that exists and has been declared, but does not yet hold a meaningful value. Using None is preferable to using 0 or "" as placeholders because it is semantically unambiguous.
# Line (12) print(type(age))	Calls the built-in type() function with age as its argument. type() returns the class (data type) of the object. The result is passed directly to print(). Output: <class 'int'> — confirming age holds an integer.
# Line (13) print(type(pi))	type(pi) returns <class 'float'> because pi was assigned a floating-point constant. This confirms Python correctly inferred the float type from the decimal point in the literal 3.14159265.
# Line (14) print(type(first_name))	type(first_name) returns <class 'str'> because first_name was assigned a string constant. In Python, even a single character like 'A' is a string (Python has no dedicated character type).
# Line (15) print(type(is_student))	type(is_student) returns <class 'bool'>. Notably, bool is a subclass of int in Python: True == 1 and False == 0 evaluate to True. This means True + True = 2 is a valid (though unusual) Python expression.
# Line (16) print(type(salary))	type(salary) returns <class 'NoneType'>. There is exactly one instance of NoneType in Python — the object None — making it a singleton.
# Line (17) print('Full name:', full_name)	Calls print() with two arguments separated by a comma. Python's print() automatically separates multiple arguments with a single space by default. Output: Full name: Arjun Sharma.
# Line (18) print('Year of birth:', year_of_birth)	Prints the string label and the integer value year_of_birth. Note that print() automatically converts non-string arguments to their string representation for display — there is no need to call str(year_of_birth) explicitly when passing to print().

Cell 2: Numeric Operators, Precedence, and Type Conversion

This cell systematically exercises all six arithmetic operators, demonstrates operator precedence rules, and illustrates type conversion between int, float, and str.

```
# — Cell 2: Numeric Operators and Type Conversion —————

# --- 2.1: Basic arithmetic operators ---
a = 17
b = 5

print('Addition:      ', a + b)      # Line (1) → 22
print('Subtraction:   ', a - b)      # Line (2) → 12
print('Multiplication: ', a * b)      # Line (3) → 85
print('Division:      ', a / b)      # Line (4) → 3.4
print('Floor Division: ', a // b)     # Line (5) → 3
print('Remainder:     ', a % b)      # Line (6) → 2
print('Power:         ', a ** b)     # Line (7) → 1419857

# --- 2.2: Operator precedence ---
expr1 = 2 + 3 * 4                    # Line (8)
expr2 = (2 + 3) * 4                  # Line (9)
expr3 = 2 ** 3 + 1                   # Line (10)
expr4 = 10 - 4 - 2                   # Line (11)

print('2 + 3 * 4      =', expr1)     # → 14
print('(2+3) * 4      =', expr2)     # → 20
print('2**3 + 1       =', expr3)     # → 9
print('10 - 4 - 2     =', expr4)     # → 4

# --- 2.3: Mixed type arithmetic ---
int_val = 10
float_val = 3.0
result = int_val + float_val        # Line (12)
print(type(result), result)         # Line (13)

# --- 2.4: Explicit type conversion ---
x = 42                               # Line (14)
x_as_float = float(x)               # Line (15)
print(x_as_float, type(x_as_float)) # → 42.0 <class 'float'>

y = 3.99                             # Line (16)
y_as_int = int(y)                   # Line (17)
```

```

print(y_as_int, type(y_as_int)) # → 3 <class 'int'> (truncates, not rounds!)

# --- 2.5: String-to-number conversion ---
sval = '123' # Line (18)
ival = int(sval) # Line (19)
print(ival + 1) # Line (20) → 124

# --- 2.6: Conversion error handling ---
nsv = 'hello bob' # Line (21)
try:
    niv = int(nsv) # Line (22)
except ValueError as e: # Line (23)
    print('Conversion error:', e) # Line (24)

```

Line-by-Line Explanation — Cell 2

Code Line	Explanation
# Line (1) <code>a + b</code>	The + operator adds two integer objects. $17 + 5 = 22$. Since both operands are integers, the result is also an integer. <code>print()</code> receives two arguments: the string label and the integer result, which it displays separated by a space.
# Line (2) <code>a - b</code>	The - operator performs subtraction. $17 - 5 = 12$. Result type is int.
# Line (3) <code>a * b</code>	The * operator performs multiplication. $17 * 5 = 85$. Result type is int.
# Line (4) <code>a / b</code>	The / operator performs true division in Python 3. Even though 17 and 5 are both integers, / always returns a float. $17 / 5 = 3.4$, not 3. This is a critical difference from Python 2.
# Line (5) <code>a // b</code>	The // operator performs floor division — it computes the quotient and discards the fractional part (floors toward negative infinity). $17 // 5 = 3$. The result is an int when both operands are integers.
# Line (6) <code>a % b</code>	The % operator computes the remainder (modulo). $17 = 5 * 3 + 2$, so $17 \% 5 = 2$. Modulo is extremely useful for checking divisibility ($n \% 2 == 0$ checks if n is even) and for wrapping index values.
# Line (7) <code>a ** b</code>	The ** operator performs exponentiation. $17 ** 5 = 17 \times 17 \times 17 \times 17 \times 17 = 1,419,857$. Python integers support arbitrarily large exponentiation without overflow — unlike most other languages.
# Line (8) <code>expr1 = 2 + 3 * 4</code>	Tests operator precedence. * has higher precedence than +, so Python evaluates $3 * 4 = 12$ first, then $2 + 12 = 14$. Result: 14 (not 20).
# Line (9) <code>expr2 = (2 + 3) * 4</code>	Parentheses override default precedence. Python evaluates $(2 + 3) = 5$ first, then $5 * 4 = 20$. This demonstrates that parentheses are the most powerful tool for controlling evaluation order.
# Line (10) <code>expr3 = 2 ** 3 + 1</code>	** has higher precedence than +. Python evaluates $2 ** 3 = 8$ first, then $8 + 1 = 9$. Result: 9 (not $2 ** 4 = 16$).

Code Line	Explanation
# Line (11) <code>expr4 = 10 - 4 - 2</code>	When operators of equal precedence appear in sequence, Python evaluates left-to-right. So: $(10 - 4) - 2 = 6 - 2 = 4$. (For <code>-</code> and <code>/</code> , left-to-right is critical because these operations are not commutative or associative in general.)
# Line (12) <code>result = int_val + float_val</code>	Mixed-type arithmetic: adding an int (10) and a float (3.0). Python's implicit type promotion rules convert the int to a float before performing addition. The result (13.0) is a float. Python never silently loses precision in this direction.
# Line (13) <code>print(type(result), result)</code>	Confirms the result's type is float: output is <code><class 'float'> 13.0</code> . The <code>.0</code> suffix confirms it is a float, not an integer.
# Line (14) <code>x = 42</code>	Assigns the integer 42 to x. This will be the source value for the explicit type conversion demonstrated on the next line.
# Line (15) <code>x_as_float = float(x)</code>	Explicitly converts the integer 42 to the float 42.0 using the <code>float()</code> built-in. This is an explicit (programmer-initiated) type conversion — Python would not have done this automatically without an arithmetic expression requiring it.
# Line (16) <code>y = 3.99</code>	Assigns the float 3.99 to y. This will be converted to an integer on the next line.
# Line (17) <code>y_as_int = int(y)</code>	Converts the float 3.99 to an integer using <code>int()</code> . CRITICAL: <code>int()</code> truncates toward zero — it discards the fractional part entirely without rounding. <code>int(3.99) = 3</code> , not 4. <code>int(-3.99) = -3</code> , not -4. Use <code>round()</code> if rounding is required.
# Line (18) <code>sval = '123'</code>	Assigns the string '123' — three characters: '1', '2', '3'. This is not the integer 123; it is a text representation of that number. Attempting arithmetic on sval directly would raise a <code>TypeError</code> .
# Line (19) <code>ival = int(sval)</code>	Converts the string '123' to the integer 123 using <code>int()</code> . This works because '123' contains only digit characters. <code>int()</code> can parse strings representing valid integer literals, including negative integers like '-42'.
# Line (20) <code>print(ival + 1)</code>	Demonstrates that ival is now a genuine integer: <code>ival + 1 = 124</code> . If we had accidentally done <code>sval + 1</code> , Python would have raised a <code>TypeError</code> ('can only concatenate str to str, not int').
# Line (21) <code>nsv = 'hello bob'</code>	Assigns a string containing non-numeric characters. Attempting to convert this with <code>int()</code> will raise a <code>ValueError</code> .
# Line (22) <code>niv = int(nsv)</code>	<code>int('hello bob')</code> raises a <code>ValueError</code> because 'hello bob' cannot be interpreted as a valid integer literal. Without the surrounding try-except, this would terminate the program with an unhandled exception.
# Line (23) <code>except ValueError as e:</code>	Catches the <code>ValueError</code> raised on Line (22). The <code>as e</code> clause binds the exception object to the name <code>e</code> , making the exception's message accessible in the except block.
# Line (24) <code>print('Conversion error:', e)</code>	Prints a descriptive error message including the exception details. This is the standard pattern for graceful error handling: inform the user of what went wrong without crashing the program.

Cell 3: The `input()` Function and Comments

This cell demonstrates runtime user input, the critical requirement to convert `input()` results before arithmetic, and the correct use of comments for documentation.

```

# — Cell 3: Input Function and Comments —————

# This program calculates the area of a rectangle.
# The user provides the dimensions at runtime.
# Formula: Area = length × width

# --- 3.1: Basic input ---
user_name = input('Enter your name: ')    # Line (1)
print('Hello,', user_name)                # Line (2)

# --- 3.2: Numeric input with conversion ---
length_str = input('Enter length (m): ')  # Line (3)
width_str  = input('Enter width  (m): ')  # Line (4)

# Without conversion, these are strings — we cannot multiply them
length = float(length_str)                # Line (5)
width  = float(width_str)                  # Line (6)

area = length * width                      # Line (7)

# Using an f-string for formatted output
print(f'Area of rectangle: {area:.2f} square metres') # Line (8)

# --- 3.3: Chaining input and conversion ---
# A more concise style: conversion applied directly on the input call
temperature_c = float(input('Enter temperature in Celsius: ')) # Line (9)
temperature_f = (temperature_c * 9 / 5) + 32                    # Line (10)
print(f'{temperature_c}°C = {temperature_f:.1f}°F')            # Line (11)

```

Line-by-Line Explanation — Cell 3

Code Line	Explanation
# Lines (1-4): Comments	Lines beginning with # are comment lines. Python's interpreter ignores them completely during execution. These comments communicate the purpose of the program, the user's role, and the mathematical formula — information that helps any reader understand the code's intent before examining the details.
# Line (1) user_name = input(...)	Calls the built-in input() function with the string 'Enter your name: ' as the prompt argument. Python prints the prompt, pauses execution, and waits for the user to type a response and press Enter. The typed response (as a string) is assigned to user_name. The trailing space in the prompt string ensures a visual gap between the prompt and the cursor.

Code Line	Explanation
# Line (2) print('Hello,', user_name)	Prints a greeting using the value the user entered. print() automatically adds a space between its multiple arguments. If the user typed 'Priya', output is: Hello, Priya.
# Line (3) length_str = input(...)	Prompts the user to enter a length value. Regardless of whether the user types '5', '3.7', or any other numeric representation, input() returns it as a string. The variable is named with the _str suffix to make this fact explicit — a self-documenting naming convention.
# Line (4) width_str = input(...)	Similarly prompts for and stores the width as a string. At this point, both length_str and width_str are string objects and cannot be multiplied directly.
# Line (5) length = float(length_str)	Converts the string length_str to a float using the float() built-in. This creates a new float object (e.g., 5.0 from '5' or 3.7 from '3.7') and assigns it to the variable length. If the user had entered a non-numeric string, float() would raise a ValueError.
# Line (6) width = float(width_str)	Same conversion for the width. Both length and width are now float objects ready for arithmetic operations.
# Line (7) area = length * width	Computes the rectangle's area as the product of two floats. The result is assigned to area. This demonstrates why type conversion is essential: without Lines (5) and (6), this line would raise a TypeError ('can't multiply sequence by non-int of type str').
# Line (8) print(f'Area: {area:.2f}...')	Uses an f-string (formatted string literal, prefixed with f). Inside the f-string, expressions within {} are evaluated and inserted. The format specifier :.2f formats area as a float with exactly 2 decimal places. For area = 3.7 * 5.0 = 18.5, output is: Area of rectangle: 18.50 square metres.
# Line (9) temperature_c = float(input(...))	Demonstrates method chaining: input() is called first (inner), its string return value is passed directly to float() (outer), and the resulting float is assigned to temperature_c. This single-line idiom is idiomatic Python for numeric input.
# Line (10) temperature_f = ...	Applies the Celsius-to-Fahrenheit conversion formula: $F = (C \times 9/5) + 32$. Python evaluates this left-to-right with appropriate precedence: (temperature_c * 9 / 5) is computed first (multiplication before addition), then + 32 is applied. Note that 9/5 performs true division, yielding 1.8.
# Line (11) print(f'{temp_c}°C = {temp_f:.1f}°F')	Uses an f-string with two interpolations. {temperature_c} inserts the Celsius value without additional formatting. {temperature_f:.1f} inserts the Fahrenheit value rounded to 1 decimal place. The ° character is a Unicode character for the degree symbol.

Cell 4: Conditional Statements — if, elif, else, try-except

This cell demonstrates all forms of conditional control flow: single-branch if, two-way if-else, multi-way if-elif-else, nested conditionals, and try-except for exception handling.

```
# — Cell 4: Conditional Statements —

# --- 4.1: Simple if statement ---
```



```

x = 5
if x < 10:                                # Line (1)
    print('x is less than 10')           # Line (2)
if x > 20:                                # Line (3)
    print('x is greater than 20')        # Line (4) - this block is skipped
print('Program continues...')            # Line (5)

# --- 4.2: Two-way if-else ---
score = 72
if score >= 60:                           # Line (6)
    print('Result: PASS')                # Line (7)
else:                                     # Line (8)
    print('Result: FAIL')                # Line (9)

# --- 4.3: Multi-way if-elif-else ---
marks = 85
if marks >= 90:                           # Line (10)
    grade = 'A'
elif marks >= 75:                         # Line (11)
    grade = 'B'
elif marks >= 60:                         # Line (12)
    grade = 'C'
elif marks >= 45:                         # Line (13)
    grade = 'D'
else:                                     # Line (14)
    grade = 'F'
print(f'Marks: {marks}, Grade: {grade}')  # → Marks: 85, Grade: B

# --- 4.4: Nested conditional ---
age = 20
has_id = True
if age >= 18:                             # Line (15)
    if has_id:                            # Line (16)
        print('Access granted: adult with ID')
    else:
        print('Access denied: no ID presented')
else:
    print('Access denied: under 18')

# --- 4.5: try-except for safe type conversion ---
raw_value = input('Enter a number: ')    # Line (17)
try:

```

```

number = float(raw_value)          # Line (18)
squared = number ** 2              # Line (19)
print(f'{number} squared = {squared}')
except ValueError:                  # Line (20)
    print(f'Error: "{raw_value}" is not a valid number.')
```

Line-by-Line Explanation — Cell 4

Code Line	Explanation
# Line (1) if x < 10:	The if keyword begins a conditional statement. The condition <code>x < 10</code> is evaluated: since <code>x = 5</code> , this evaluates to <code>True</code> . The colon is mandatory — it signals the start of the indented block. Because the condition is <code>True</code> , Python will execute the indented lines that follow.
# Line (2) print('x is ...')	This line is indented by 4 spaces, placing it inside the if block. Because the if condition evaluated to <code>True</code> , this statement executes. Output: <code>x is less than 10</code> .
# Line (3) if x > 20:	A second, independent if statement. The condition <code>x > 20</code> evaluates to <code>False</code> (since <code>x = 5</code>). This is a separate if, not an <code>elif</code> — Python will evaluate this condition regardless of the first if's outcome.
# Line (4) print('x is ...')	This statement is inside the second if block. Because the condition on Line (3) is <code>False</code> , this statement is skipped entirely. Python's execution jumps past the indented block.
# Line (5) print('Program continues...')	This statement is at the base indentation level — it is outside both if blocks. It executes unconditionally, demonstrating that code after the if statements always runs regardless of the conditions.
# Line (6) if score >= 60:	Evaluates <code>score >= 60</code> . With <code>score = 72</code> , this is <code>True</code> ($72 \geq 60$). The <code>>=</code> operator returns <code>True</code> when the left operand is greater than or equal to the right.
# Line (7) print('Result: PASS')	Executes because the if condition is <code>True</code> . Output: <code>Result: PASS</code> .
# Line (8) else:	The else clause is paired with the if on Line (6). It has no condition of its own. Its block executes only if the if condition is <code>False</code> . Since the if was <code>True</code> , this else block is skipped.
# Line (9) print('Result: FAIL')	Inside the else block. Skipped because the if condition was <code>True</code> . In a two-way if-else, exactly one block always executes.
# Line (10) if marks >= 90:	Begins the multi-way <code>elif</code> chain. Evaluates <code>marks >= 90</code> : <code>False</code> ($85 < 90$). Python proceeds to evaluate the first <code>elif</code> .
# Line (11) elif marks >= 75:	Evaluates <code>marks >= 75</code> : <code>True</code> ($85 \geq 75$). Python executes the associated block (<code>grade = 'B'</code>) and then skips all remaining <code>elif</code> and <code>else</code> clauses without evaluating them.
# Line (12-14) elif / else	These branches are not evaluated because Line (11)'s condition was <code>True</code> . This is the key efficiency property of <code>elif</code> chains: Python stops evaluating as soon as it finds a <code>True</code> condition.
# Line (15) if age >= 18:	Outer if of the nested conditional. With <code>age = 20</code> , evaluates to <code>True</code> . Python enters the outer if block and encounters another if statement inside it.

Code Line	Explanation
# Line (16) if has_id:	Inner if — nested inside the outer if block. has_id is True (a boolean variable). Python evaluates its truthiness: True → executes the inner if block. Output: Access granted: adult with ID. This demonstrates two-level decision making where the inner condition is only relevant if the outer condition is satisfied.
# Line (17) raw_value = input(...)	Pauses execution and reads user input as a string. The variable name raw_value signals that the value has not yet been validated or converted — it is raw text from the user.
# Line (18) number = float(raw_value)	Inside the try block. Attempts to convert raw_value to a float. If the user entered '3.5', this succeeds and number = 3.5. If the user entered 'abc', float('abc') raises a ValueError.
# Line (19) squared = number ** 2	Computes the square. Only reached if Line (18) did not raise an exception. The ** operator raises number to the power 2.
# Line (20) except ValueError:	Catches any ValueError raised within the try block. This is more specific than a bare except, which would catch all exceptions including KeyboardInterrupt. Best practice is to catch the most specific exception type applicable to the error being handled.

Cell 5: Defining and Calling Functions

This cell defines several functions of increasing complexity, demonstrating parameters, return values, the scope of local variables, and the difference between functions that return values and those that produce side effects.

```
# — Cell 5: Functions —————

# --- 5.1: Function with no parameters and no return value ---
def greet():                                # Line (1)
    print('Hello from a function!')         # Line (2)

greet()                                     # Line (3)

# --- 5.2: Function with parameters ---
def greet_person(name):                     # Line (4)
    print('Hello,', name + '!!')           # Line (5)

greet_person('Priya')                       # Line (6)
greet_person('Rahul')                       # Line (7)

# --- 5.3: Function with return value ---
def square(n):                              # Line (8)
    result = n ** 2                         # Line (9)
```

return result	# Line (10)
sq = square(7)	# Line (11)
print('7 squared is:', sq)	# Line (12) → 7 squared is: 49
# --- 5.4: Function with multiple parameters ---	
def calculate_area(length, width):	# Line (13)
area = length * width	# Line (14)
return area	# Line (15)
room_area = calculate_area(5.5, 3.2)	# Line (16)
print(f'Room area: {room_area:.2f} sq m')	
# --- 5.5: Function calling another function ---	
def celsius_to_fahrenheit(c):	# Line (17)
return (c * 9 / 5) + 32	# Line (18)
def print_weather_report(city, temp_c):	# Line (19)
temp_f = celsius_to_fahrenheit(temp_c)	# Line (20)
print(f'{city}: {temp_c}°C / {temp_f:.1f}°F')	# Line (21)
print_weather_report('Delhi', 38)	# Line (22)
print_weather_report('Shimla', 12)	# Line (23)
# --- 5.6: Demonstrating None return ---	
def no_return():	# Line (24)
x = 10	# does some work but returns nothing
result = no_return()	# Line (25)
print('Return value of no_return():', result)	# → None

Line-by-Line Explanation — Cell 5

Code Line	Explanation
# Line (1) def greet():	The def keyword begins a function definition. greet is the function's name. The empty parentheses () indicate this function accepts no parameters. The colon terminates the header and signals the indented body block follows. At this point, Python stores the function definition but does not execute any of its body statements.
# Line (2) print('Hello from...')	This line is indented by 4 spaces — it is the body of the greet function. It will only execute when greet() is called. Calling print() here is a side

Code Line	Explanation
	effect: the function produces observable behaviour (console output) but does not return a value.
# Line (3) greet()	The function call. Python looks up the name 'greet', finds the function object defined on Line (1), transfers execution into the function body, executes Line (2), then returns to the calling context. Output: Hello from a function!
# Line (4) def greet_person(name):	Defines a function with one parameter: name. A parameter is a local variable that will be bound to the argument value supplied at call time. Each call to greet_person() creates a fresh binding for name.
# Line (5) print('Hello,', name + '!')	Uses the parameter name inside the function body. name + '!' concatenates the parameter's value with the exclamation mark string. When called with 'Priya', prints: Hello, Priya!
# Line (6) greet_person('Priya')	Calls greet_person with the argument 'Priya'. Inside the function, the parameter name is bound to 'Priya'. Output: Hello, Priya!
# Line (7) greet_person('Rahul')	Calls the same function again with a different argument. Demonstrates reusability: the same function serves both calls without any code duplication. Output: Hello, Rahul!
# Line (8) def square(n):	Defines a function that accepts one parameter n. This function computes a value and returns it — unlike greet(), which only produced a side effect.
# Line (9) result = n ** 2	Computes n squared using the ** exponentiation operator and stores the result in a local variable named result. Local variables exist only within the function's scope — result is not accessible outside square().
# Line (10) return result	The return statement sends the value of result back to the caller. Execution of the function terminates at this point. The calling expression square(7) evaluates to whatever result contains — in this case, 49.
# Line (11) sq = square(7)	Calls square() with argument 7. Inside the function, n = 7, result = 49, and return result passes 49 back. The expression square(7) evaluates to 49, which is then assigned to sq.
# Line (12) print('7 squared is:', sq)	Prints the result stored in sq. Output: 7 squared is: 49.
# Line (13) def calculate_area(length, width):	Defines a function with two parameters: length and width. Multiple parameters are separated by commas in both the definition (parameter list) and the call (argument list).
# Line (14) area = length * width	Multiplies the two parameter values and stores the result in the local variable area. If length = 5.5 and width = 3.2, area = 17.6.
# Line (15) return area	Returns the computed area to the caller.
# Line (16) room_area = calculate_area(5.5, 3.2)	Calls calculate_area with two positional arguments. Python binds length to 5.5 and width to 3.2 in the order they appear. The return value (17.6) is assigned to room_area.
# Line (17) def celsius_to_fahrenheit(c):	Defines a conversion utility function. This function encapsulates a specific formula, making it reusable and testable independently.

Code Line	Explanation
# Line (18) return (c * 9 / 5) + 32	Computes and returns the Fahrenheit equivalent in a single expression without storing an intermediate variable. This is valid because the return statement can accept any expression.
# Line (19) def print_weather_report(city, temp_c):	Defines a higher-level function that calls celsius_to_fahrenheit() internally. This demonstrates function composition: complex behavior built by combining simpler functions.
# Line (20) temp_f = celsius_to_fahrenheit(temp_c)	Calls celsius_to_fahrenheit with temp_c as the argument. This is one function calling another. The result (a float) is stored in the local variable temp_f.
# Line (21) print(f'{city}: ...')	Formats and prints the weather report using an f-string with both city and the temperature values.
# Line (22-23) print_weather_report calls	Demonstrates reuse: the same function handles both Delhi at 38°C and Shimla at 12°C without code repetition.
# Line (24) def no_return():	Defines a function that does internal work but has no return statement.
# Line (25) result = no_return()	Calls no_return() and assigns its return value to result. Since no_return() has no return statement, Python implicitly returns None. Result: result = None. This demonstrates the importance of always having a return statement when the caller expects a value.

Cell 6: Loops and Iteration — while, for, break, continue

This cell implements all loop types and demonstrates the five canonical loop patterns: counting, summation, maximum, minimum, and boolean search.

```
# — Cell 6: Loops and Iteration —————

# --- 6.1: Basic while loop ---
n = 1                                # Line (1)
while n <= 5:                         # Line (2)
    print(n, end=' ')                # Line (3)
    n = n + 1                        # Line (4)
print()                               # Line (5) # newline after loop
# Output: 1 2 3 4 5

# --- 6.2: Zero trip loop ---
count = 0                             # Line (6)
while count > 0:                       # Line (7) — condition False immediately
    print('This never executes')      # Line (8) — skipped entirely
print('Zero trip loop complete.')     # Line (9)

# --- 6.3: break and continue ---
```

```

print('--- break demo ---')
while True:                                # Line (10) - intentional infinite loop
    line = input('Enter text ("done" to quit): ') # Line (11)
    if line[0] == '#':                        # Line (12)
        continue                             # Line (13)
    if line == 'done':                        # Line (14)
        break                                 # Line (15)
    print('You entered:', line)               # Line (16)
print('Exited the loop.')                    # Line (17)

# --- 6.4: for loop with list ---
numbers = [9, 41, 12, 3, 74, 15] # Line (18)
print('--- for loop ---')
for num in numbers:                          # Line (19)
    print(num, end=' ')
print()

# --- 6.5: for loop with range() ---
for i in range(1, 6):                        # Line (20)
    print(i, end=' ')
print() # Output: 1 2 3 4 5

# --- 6.6: Counting pattern ---
count = 0                                    # Line (21)
for num in numbers:
    count = count + 1                        # Line (22)
print('Count:', count)                      # → Count: 6

# --- 6.7: Summation pattern ---
total = 0                                    # Line (23)
for num in numbers:
    total = total + num                     # Line (24)
print('Sum:', total)                        # → Sum: 154

# --- 6.8: Maximum pattern ---
largest = -1                                 # Line (25)
for num in numbers:
    if num > largest:                         # Line (26)
        largest = num                       # Line (27)
print('Largest:', largest)                  # → Largest: 74

# --- 6.9: Minimum pattern (with None sentinel) ---

```

```

smallest = None                                # Line (28)
for num in numbers:
    if smallest is None:                        # Line (29)
        smallest = num
    elif num < smallest:                        # Line (30)
        smallest = num
print('Smallest:', smallest)                   # → Smallest: 3

# --- 6.10: Boolean search pattern ---
target = 3
found = False                                # Line (31)
for num in numbers:
    if num == target:                          # Line (32)
        found = True
print(f'Value {target} found: {found}')        # → Value 3 found: True

```

Line-by-Line Explanation — Cell 6

Code Line	Explanation
# Line (1) <code>n = 1</code>	Initializes the iteration variable <code>n</code> to 1. This is the starting value for the count. The while loop's body will increment <code>n</code> after each iteration, ensuring the loop eventually terminates.
# Line (2) <code>while n <= 5:</code>	The while statement's condition. Python evaluates <code>n <= 5</code> before each iteration. Initially <code>1 <= 5</code> is True. The loop continues until <code>n</code> becomes 6, at which point <code>6 <= 5</code> is False and the loop terminates.
# Line (3) <code>print(n, end=' ')</code>	Prints the current value of <code>n</code> . The <code>end=' '</code> keyword argument overrides <code>print()</code> 's default behaviour of appending a newline (<code>end='\n'</code>) — instead, a space is printed after each value, keeping all values on the same line.
# Line (4) <code>n = n + 1</code>	The increment statement — the most critical line in the loop. It reads the current value of <code>n</code> , adds 1, and stores the result back into <code>n</code> . Without this line, <code>n</code> would remain 1 forever and the loop would be infinite.
# Line (5) <code>print()</code>	Calling <code>print()</code> with no arguments outputs a single newline character, moving the cursor to the next line after all the loop output. This is a common idiom when using <code>end=' '</code> inside a loop.
# Line (6-7) <code>count = 0; while count > 0:</code>	Demonstrates the zero-trip loop. <code>count = 0</code> is initialized, and the condition <code>count > 0</code> evaluates to False (<code>0 > 0</code> is False) immediately. The loop body is never entered. This is valid and intentional behaviour — while loops do not guarantee at least one execution.
# Line (10) <code>while True:</code>	Creates an intentionally infinite loop — <code>while True</code> always has a True condition. The only way to exit is via a <code>break</code> statement inside the body. This pattern is common for interactive input loops where the exit condition can only be tested after reading input.
# Line (11) <code>line = input(...)</code>	Reads a line of user input inside the loop. The result is stored in <code>line</code> as a string.

Code Line	Explanation
# Line (12) if line[0] == '#':	Checks whether the first character (index 0) of the input string is a hash symbol. String indexing in Python uses square brackets; indices start at 0.
# Line (13) continue	If the input starts with '#', continue immediately ends the current iteration and returns to the top of the while loop — re-checking the condition (always True) and reading the next input. Lines below continue (including Line 16) are not executed for this iteration.
# Line (14-15) if line == 'done': break	Checks for the termination keyword. If the user typed exactly 'done', break exits the while loop immediately. Execution resumes at Line (17) — the first statement after the loop.
# Line (16) print('You entered:', line)	Only reached if the input neither starts with '#' nor equals 'done'. This is the normal processing path for valid input.
# Line (17) print('Exited the loop.')	Executes after the loop terminates via break. Demonstrates that execution continues normally after a break.
# Line (18) numbers = [9, 41, 12, 3, 74, 15]	Defines a Python list containing six integer elements. A list is an ordered, mutable sequence of items enclosed in square brackets and separated by commas. This list will serve as the iterable for the subsequent for loops.
# Line (19) for num in numbers:	Begins a for loop that iterates over the list numbers. On each iteration, num is automatically assigned the next element from the list: first 9, then 41, then 12, and so on. No index management or increment statement is required.
# Line (20) for i in range(1, 6):	range(1, 6) generates the sequence 1, 2, 3, 4, 5 (the stop value 6 is excluded). On each iteration, i is assigned the next value from this sequence. range() is the standard tool for generating numeric sequences in Python.
# Line (21) count = 0	Initializes the counter variable to 0 before the loop. This is the accumulator pattern. The counter must be initialized outside the loop; initializing it inside would reset it to 0 on every iteration, defeating the purpose.
# Line (22) count = count + 1	Increments the counter by 1 on every iteration, regardless of the element's value. After all 6 elements are processed, count = 6.
# Line (23-24) total = 0; total = total + num	Implements the summation pattern. total starts at 0 and accumulates the sum of all elements. After processing all 6 elements: 0+9+41+12+3+74+15 = 154.
# Line (25) largest = -1	Initializes the tracking variable for the maximum. Using -1 works here because all values in numbers are positive. For general use, initialize to the first element or to float('-inf') (negative infinity).
# Line (26-27) if num > largest: largest = num	Compares the current element to the running maximum. If the current element is larger, it becomes the new maximum. After processing [9,41,12,3,74,15]: 9→41→74 (41 is replaced by 74 when 74 is encountered).
# Line (28) smallest = None	Initializes the tracking variable to None rather than a specific number. None serves as a sentinel value indicating that no element has been processed yet.
# Line (29) if smallest is None:	Tests whether this is the first iteration (no element processed yet). The is operator tests object identity — None is a singleton, so this check is correct and idiomatic. If True, sets smallest to the current element unconditionally.
# Line (30) elif num < smallest:	On all subsequent iterations (smallest is no longer None), checks whether the current element is smaller than the running minimum. If True, updates smallest.

Code Line	Explanation
# Line (31) found = False	Initializes the boolean flag to False. The flag communicates whether the search was successful.
# Line (32) if num == target:	Checks whether the current element equals the target value (3). If True, sets found = True. Note: the loop does not break after finding the value — it continues checking remaining elements, which is unnecessary but correct. A more efficient version would include a break after found = True.

Chapter Summary and Key Takeaways

KEY TAKEAWAYS — Course 1: Getting Started with Python

1. Python is a high-level, dynamically typed, interpreted language whose syntax prioritizes readability.

Reserved words are the grammar; variables are the vocabulary; statements are the sentences.

2. Variables are dynamically typed — the same name can hold values of different types at different times.

Variable naming follows strict rules (must start with letter or underscore; case-sensitive).

3. Python 3's / operator always returns a float; use // for integer (floor) division.

Operator precedence follows PEMDAS. Use parentheses liberally to clarify intent.

4. Type conversion is explicit: int(), float(), str(). input() ALWAYS returns a string.

Wrap conversions in try-except to handle invalid user input gracefully.

5. Indentation is syntactically mandatory in Python — it defines code block scope.

Use 4 spaces per level (PEP 8). Never mix tabs and spaces.

6. if-elif-else chains evaluate conditions top-to-bottom; only the first True branch executes.

Nested conditionals express compound logic where inner conditions depend on outer ones.

7. Functions encapsulate reusable logic. def defines; calling executes. Parameters receive

arguments. `return` sends a value back. A function without `return` implicitly returns `None`.

8. `while` loops are indefinite; `for` loops are definite. `break` exits; `continue` skips to next iteration. Always ensure `while` loops have a termination condition to avoid infinite execution.

9. Mastering five loop patterns — counting, summation, maximum, minimum, boolean search — enables the implementation of a vast range of computational algorithms.

What's Next

Course 2 will introduce Python's built-in collection types — Strings, Lists, Dictionaries, and Tuples — and examine the powerful methods Python provides for working with these data structures. These collections, combined with the control flow constructs mastered in this chapter, form the foundation for all data-intensive Python programming.